

Data Transfer and Addressing

Memory operands, load/store widths, addressing modes, alignment, and data traversal

Computer Organization & Architecture | AArch64 Reference

Topic 6 Outline

Topic Objective

Explain how AArch64 transfers values between registers and memory and how an instruction forms the effective address for a memory access.

- **Section 1:** Memory addresses and load/store semantics
- **Section 2:** Transfer width and extension
- **Section 3:** AArch64 addressing modes
- **Section 4:** Arrays, pointers, and byte data
- **Section 5:** Memory alignment and pair transfers
- **Section 6:** Summary and self-test

Memory Addresses and Load/Store Semantics

AArch64 Is a Load/Store Architecture

Separation of Memory Access and Computation

Arithmetic and logical instructions operate on register values. Dedicated load and store instructions transfer values between registers and memory.

```
ldr x1, [x0]
add x1, x1, #5
str x1, [x0]
```

- `ldr` reads a value from memory into a register.
- `add` changes the register value.
- `str` writes the register value back to memory.

Memory Is Byte Addressed

- Each memory address identifies one byte.
- Multi-byte values occupy consecutive addresses.
- The instruction determines how many bytes are transferred.
- The base address for a load/store is held in a 64-bit base register or in `sp`.

Example

A 64-bit value beginning at `0x1000` occupies byte addresses `0x1000` through `0x1007`.

Address	Byte
<code>0x1000</code>	byte 0
<code>0x1001</code>	byte 1
<code>0x1002</code>	byte 2
	:
<code>0x1007</code>	byte 7

Bracket Notation Means Memory Access

Register Operand

```
add x0, x1, x2
```

- Uses values already in registers.
- No data-memory operand appears.

Memory Operand

```
ldr x0, [x1]
```

- x1 contains an address.
- Brackets mean access memory at that address.

Do Not Confuse Address and Data

The value in x1 is the address. The bytes stored at that address are the data transferred by the load or store.

Loading Data with `ldr`

64-Bit Load

```
ldr x0, [x1]
```

- The effective address is the 64-bit value in `x1`.
- Eight bytes are read from memory.
- The 64-bit value is written into `x0`.

32-Bit Load

`ldr w0, [x1]` reads four bytes into `w0`. Any write to `w0` clears bits `[63:32]` of the corresponding `x0` register.

Storing Data with `str`

64-Bit Store

```
str x0, [x1]
```

- The effective address is the value in `x1`.
- The eight-byte value in `x0` is written to memory.
- The source register is not changed by the store.

Narrow Stores

`strb` writes the low 8 bits of a register, and `strh` writes the low 16 bits. `str w0, [x1]` writes the low 32 bits.

Transfer Width and Extension

Load Width Determines Bytes Read

Instruction	Bytes Read	Destination Result
<code>ldrb w0, [x1]</code>	1 byte	Zero-extended to 32 bits in w0
<code>ldrh w0, [x1]</code>	2 bytes	Zero-extended to 32 bits in w0
<code>ldr w0, [x1]</code>	4 bytes	Written to w0; upper half of x0 becomes zero
<code>ldr x0, [x1]</code>	8 bytes	Full 64-bit value written to x0

Width Comes from the Instruction

A memory address does not carry a type. The load/store instruction determines the number of bytes transferred.

Signed Loads Preserve Negative Values

Sign Extension

A signed load copies the sign bit of the narrow source into the new upper bits so the two's-complement value is preserved.

Instruction	Source Width	Result
<code>ldrsb x0, [x1]</code>	8 bits	Sign-extended to 64 bits
<code>ldrsh x0, [x1]</code>	16 bits	Sign-extended to 64 bits
<code>ldrsx x0, [x1]</code>	32 bits	Sign-extended to 64 bits

Example

If memory contains the byte `0xff`, `ldrb` interprets the transferred bits as 255 after zero extension, while `ldrsb x0, [x1]` produces the 64-bit representation of `-1`.

Load Width and Extension Example

Assume memory at 0x2000 contains the byte 0x80.

Unsigned Byte Load

```
ldrb w0, [x1] w0 =  
0x00000080
```

- Decimal value: 128
- Upper bits filled with zeros

Signed Byte Load

```
ldrsh x0, [x1] x0 =  
0xffffffffffff80
```

- Decimal value: -128
- Sign bit replicated upward

AArch64 Addressing Modes

The Effective Address

Definition

The effective address is the memory address computed for a particular load or store instruction.

- A base register supplies the starting address.
- An optional immediate or register offset can modify that address.
- Some addressing forms also write an updated address back to the base register.

Base Register

AArch64 memory operands normally use a 64-bit general-purpose register or `sp` as the base. A `w` register is not used as the base address register.

Base Register Addressing

Syntax

```
ldr x0, [x1]
```

- Effective address = x1.
- x1 is unchanged.
- This corresponds closely to dereferencing a pointer already holding the required address.

C Analogy

If x1 represents pointer p, then the memory operand [x1] corresponds conceptually to *p.

Immediate Offset Addressing

Example

```
ldr x0, [x1, #16]
```

- Effective address = $x1 + 16$.
- The base register $x1$ is unchanged.
- Useful for structure fields and fixed-position array elements.

Encoding Detail

A64 has more than one load/store immediate encoding. Common unsigned offsets are scaled by transfer size; unscaled forms use a smaller signed byte offset.

Register Offset Addressing

Unscaled Register Offset

`ldr x0, [x1, x2]` Effective address = $x1 + x2$.

Scaled Index for 64-Bit Elements

`ldr x0, [x1, x2, lsl #3]` Effective address = $x1 + (x2 \times 8)$.

Register-offset forms are well suited to array indexing because the index can be scaled to the element size.

Using a 32-Bit Index Register

AArch64 register-offset addressing can extend a 32-bit index while forming a 64-bit address.

Unsigned 32-Bit Index

```
ldr x0, [x1, w2, uxtw #3]
```

- w2 is zero-extended to address width.
- lsl #3 scales the index by 8 bytes.
- This is useful when the source-language array index is a 32-bit unsigned value.

Signed Index

sxtw can be used instead when the 32-bit index must be sign-extended before address formation.

Pre-Indexed Addressing

Example

```
ldr x0, [x1, #8]!
```

- 1 Compute $x1 + 8$.
- 2 Use the updated address for the memory access.
- 3 Write the updated address back into $x1$.

Syntax Marker

The exclamation mark indicates writeback. In A64, pre-indexed writeback uses an immediate offset, not a general-purpose register offset.

Post-Indexed Addressing

Example

```
ldr x0, [x1], #8
```

- 1 Access memory using the original value of `x1`.
- 2 After the access, compute `x1 + 8`.
- 3 Write the updated address back into `x1`.

Common Use

Post-indexing is useful when processing the current element and then advancing a pointer to the next element.

Addressing Mode Comparison

Form	Example	Access Address	Base After
Base	<code>ldr x0, [x1]</code>	<code>x1</code>	unchanged
Immediate offset	<code>ldr x0, [x1, #16]</code>	<code>x1 + 16</code>	unchanged
Register offset	<code>ldr x0, [x1, x2]</code>	<code>x1 + x2</code>	unchanged
Pre-index	<code>ldr x0, [x1, #8]!</code>	<code>x1 + 8</code>	<code>x1 + 8</code>
Post-index	<code>ldr x0, [x1], #8</code>	<code>x1</code>	<code>x1 + 8</code>

Key Distinction

Offset addressing computes an access address without changing the base register; pre-indexed and post-indexed forms also write an updated address back to the base.

Arrays, Pointers, and Byte Data

Array Address Calculation

For an array with element size S , element $A[i]$ begins at:

$$\text{address}(A[i]) = \text{base}(A) + i \times S$$

Example: 64-Bit Array

If $x1$ contains the array base and $x2$ contains index i :

```
ldr x0, [x1, x2, lsl #3]
```

- `lsl #3` multiplies the index by $2^3 = 8$.
- The computed byte offset selects the requested 64-bit element.

Traversing an Array with a Pointer

Assume `x0` points to the current 64-bit array element.

```
ldr x1, [x0], #8
```

- The load copies the current 64-bit element into `x1`.
- The memory access uses the original address in `x0`.
- After the load, post-index writeback advances `x0` by 8 bytes.
- `x0` therefore points to the next array element.

Pointer Traversal

Post-index addressing combines the current access with advancement to the next element. Topic 7 will place this operation inside a loop.

Byte Arrays and Pointer Advancement

A C-style string is a byte array, so one character occupies one byte.

```
ldrb w1, [x0], #1
```

- `ldrb` reads the current byte and zero-extends it into `w1`.
- The access uses the original address in `x0`.
- Post-index writeback advances `x0` by one byte.
- Testing for the null terminator and repeating the access are Topic 7 control-flow operations.

Scope Boundary

This instruction demonstrates byte access and pointer movement only; the complete string-traversal loop belongs in the next topic.

Alignment and Pair Transfers

Natural Alignment

Natural Alignment Rule

An n -byte object is naturally aligned when its starting address is a multiple of n .

- 2-byte halfword: address divisible by 2.
- 4-byte word: address divisible by 4.
- 8-byte doubleword: address divisible by 8.

AArch64 Qualification

Many ordinary AArch64 loads and stores to normal memory permit unaligned addresses, but natural alignment is preferred. Misaligned accesses may be slower or cross memory-system boundaries, and some access classes have stricter requirements.

Load and Store Pair: `ldp` and `stp`

Store Pair

```
stp x0, x1, [x2]
```

- Stores two register values.
- The memory locations are adjacent.

Load Pair

```
ldp x0, x1, [x2]
```

- Loads two register values.
- The memory locations are adjacent.

Important Qualification

A pair transfer is one architectural instruction, but it is not guaranteed to be atomic or to execute as one physical memory-bus transaction.

Summary and Self-Test

Topic 6 Summary

- AArch64 uses explicit load/store instructions to move values between registers and memory.
- The instruction determines transfer width and whether a narrow loaded value is zero-extended or sign-extended.
- Effective addresses can use a base register, immediate offset, register offset, or pre/post-indexed immediate writeback.
- Scaled register offsets map naturally to array indexing; post-indexing maps naturally to pointer traversal.
- Natural alignment is preferred, although many ordinary unaligned accesses to normal memory are permitted.
- `ldp` and `stp` transfer adjacent register pairs without implying atomicity or a particular physical bus transaction.

Self-Test 1/2

- ① What does the bracket notation in `ldr x0, [x1]` mean?
- ② How many bytes are read by `ldrb`, `ldrh`, `ldr w0, [...]`, and `ldr x0, [...]`?
- ③ What is the difference between `ldrb w0, [x1]` and `ldrsb x0, [x1]`?
- ④ For `ldr x0, [x1, #16]`, what is the effective address and what happens to `x1`?
- ⑤ Why is `ldr x0, [x1, x2, lsl #3]` suitable for a 64-bit array?

Self-Test 2/2

- ⑥ What changes in x1 after `ldr x0, [x1, #8]!?`
- ⑦ What changes in x1 after `ldr x0, [x1], #8?`
- ⑧ Why is `ldr x0, [x1, x2]!` not a valid example of A64 pre-indexed register addressing?
- ⑨ What does natural alignment mean for an 8-byte object?
- ⑩ Why does `ldrb w1, [x0], #1` advance a byte-array pointer by one rather than by eight?

Branching and Iteration

Topic 7 develops the comparison, branching, and control-flow instructions used to implement decisions and repetition.

- Comparison instructions
- Conditional and unconditional branches
- Loops and selection structures
- Translation of high-level control flow